

408 Course Atlas

四科世界模型、接口与综合路径

408 计算机综合方法论总册

母问题：408 到底在研究什么？

408 不是四本教材的并排目录。它研究的是：**数据怎样被组织，程序语义怎样落到机器，操作系统怎样管理执行与资源，独立机器又怎样通过网络通信。**

可以把观察路径压缩成：

Data → Program → Machine → Operating System → Network

但这条链只是**系统观察路径**，不是一个万能世界模型。四科必须保留自己的对象、状态和推理语言。

1 为什么不能用一种语言强行统一四科

同一道综合题会跨越多个层级，但不同学科真正关心的变量不同。若强行用“对象-状态-机制”一套抽象吞掉全部细节，很容易得到漂亮但不可操作的总图。

学科	学科母问题	首先观察什么
数据结构	怎样按 Workload 组织关系，使关键操作以可接受代价完成？	Relation / Representation / Operation / Invariant / Cost
计算机组成原理	怎样用有限硬件实现 ISA 对软件承诺的程序语义？	State / Location / Path / Resource / Timing / Commit
操作系统	怎样在并发、有限资源和异步事件下提供受保护的执行环境？	Object / Relation / Queue / Event / Mechanism / Policy
计算机网络	没有共享内存和全局即时状态时，独立机器怎样完成通信？	Scope / Name / State Owner / Event / Transition / Feedback

共同语言只用于切换镜头

四科可以共享一组跨科观察槽：

Object / State / Scope / Mapping / Resource / Event / Invariant / Cost

它们只帮助你判断当前应该切到哪门学科，不替代任何 Subject Atlas 的本体模型。

2 数据结构：从 Workload 反推表示

Data Structure Mother Model

Workload → Logical Relation → Required Operations → Representation → Invariant → Algorithm → Cost

数据结构的根本判断不是“这题属于链表还是树”，而是：当前关系是什么、要支持哪些操作、为了这些操作值得维护什么结构、维护成本是多少。

复杂度、渐进分析、Cost Vector、Workload-first 与 ‘Relation ≠ Representation’ 属于 Atlas Foundation，而不是独立机制 Topic。

核心 Topic 依次覆盖：线性关系与存储表示、栈队列、串与模式匹配、树、Heap、Union-Find、图表示与遍历、图结构算法、查找与有序索引、Hash、内部排序、外部排序。

3 计算机组成原理：从 ISA 语义到精确提交

Computer Organization Mother Model

ISA Semantic → Data Movement → Hardware Path → Timing → Architectural State Commit

硬件内部可以缓存、重叠、等待甚至进行局部推测，但最终必须保持软件可见的 ISA 语义。

本学科八个核心 Topic 是：数据表示与运算、ISA 与机器级程序、CPU 数据通路与控制、流水线、主存硬件、Cache、地址翻译硬件、总线与 I/O 硬件。

计组起手六问

1. 当前软件可见状态是什么？
2. 数据现在在哪里？
3. 它经过哪条硬件路径？
4. 需要哪些共享资源？
5. 数据何时可用，何时锁存？
6. 最终何时改变体系结构状态？

4 操作系统：在状态变化中维护安全与进展

Operating System Mother Model

OS 统一状态可写成：

$$S = (Objects, Relations, Queues)$$

推演主线是：

State → Transition → Bad State → Safety / Liveness → Minimal Mechanism → Tradeoff

OS Atlas Foundation 负责解释 abstraction、virtualization、protection、user/kernel、程序运行环境等共同入口；真正的机制 Topic 只有五个：执行实体与调度、并发同步与死锁、虚拟内存、I/O、文件系统。

不要把 Foundation 伪装成第六个机制 Topic

基础概念是后续机制共同使用的坐标系。它没有必要为了目录整齐与五个机制 Topic 平级竞争 Ownership。

5 计算机网络：没有全局即时状态的通信

Computer Network Mother Model

Scope → State → Event → Transition → Feedback → Cost

网络中的同一个词只有放回作用域才有意义：一跳、端到端、一个 AS、整个 Internet，分别由不同对象保存状态并作出决策。

八个核心 Topic 是：通信基础与性能、单跳交付、可靠传输、IP 与分组转发、路由、UDP/TCP、拥塞控制、应用层服务语义。

网络起手八问

先确认：Scope、通信对象、名字、状态所有者、事件、状态/封装变化、反馈信号、时间/带宽/队列成本。

6 Bridge：只有真实交接才值得独立建册

Bridge 不是“两个知识点有联系”。它必须先通过 Validity，再通过 Standalone Promotion。

Gate 1: 这是不是一个真实接口?

删除具体题目后，仍应存在稳定的：

$$A \text{ output} \rightarrow \text{translation/shared structure} \rightarrow B \text{ input}$$

如果只是 B 使用了 A 的既有机制，优先记为 Use；如果只是多个模块完成一个具体过程，则归 Integration。

Gate 2: 它值得独立建册吗?

继续检查四项：**Ownership Pressure / Reuse / Current-Scope Relevance / New Inference**。

真连接但没有独立维护必要，不占一个 Canonical Bridge Owner。

当前三个 Core Cross-Subject Bridge 为：

1. **X-B01 Privilege / Exception / System Call × OS Control**: 硬件控制转移怎样成为内核接管执行的入口；
2. **X-B02 Hardware Address Translation × OS Virtual Memory**: OS 建立映射，硬件消费映射，缺失时 fault 回到 OS 修复；
3. **X-B03 Interrupt / DMA × OS I/O**: 设备完成怎样从硬件事件变成内核 completion 与进程 wakeup。

X-B04 Process / Socket × Transport Endpoint 结构上是真接口，但是否晋升 Core 仍需 408 真题和 Coverage evidence 支撑。

Anti-Bridge: 几个常见误区

- Hardware Cache $\neq$ OS Page Cache;
- TLB miss $\neq$ Cache miss $\neq$ Page Fault;
- Routing algorithm uses graph ideas $\neq$ routing protocol is literally a data-structure algorithm;
- “系统里用了 Queue/Tree/Hash”通常只是 Use，不自动产生 Data Structure × Systems Bridge。

7 Integration: 完整过程怎样组合成熟模块

Integration 拥有具体协作轨迹，不重新定义局部机制。

X-I01: 一次 LOAD / Memory Access 的完整慢路径

Instruction → Effective Address → VA → TLB/Page Table → PA → Cache → Memory

如果发生 Page Fault:

Fault → Kernel → VM Repair → PTE Update → Retry → Commit

X-I02: 一次 Blocking File read 的完整生命周期

User Process → System Call → fd/OFD/inode → Page Cache / I/O Request → Block → DMA/Interrupt → C

8 做题控制：先切对学科，再调机制

综合题的第一动作

不要先问“这是哪一章”。先问：题目现在要求我追踪的是关系、硬件路径、OS 状态，还是分布式通信状态？

一旦确定当前层：

- 数据结构: Relation → Representation → Operation → Invariant → Cost;
- 计组: State → Location → Path → Timing → Commit;
- OS: Object/Queue → Event → Mechanism/Policy → New State;
- 网络: Scope/Name/Owner → Event → Transition/Feedback。

9 怎样使用这本总册

1. 第一次学习某一科时，只把本册当地图，不用先背全部跨科关系；
2. 进入对应 Subject Atlas，建立那门学科自己的世界模型；
3. 学到某个 Topic 时，先掌握局部机制；
4. 只有两个 Owner 真正交换状态、数据或控制权时，打开 Bridge；
5. 只有要追踪一个完整真实过程时，打开 Integration；
6. 做完题以后，把失误归到模型、识别、路径、执行/检查/表达或考试决策，而不是无限扩张知识分类。

一页压缩

408 的主干不是“记住四科所有章节”，而是：

Separate Subject Models + Explicit Interfaces + Canonical Processes + Executable Control

四科先各自讲清，接口只在真实交接处出现，综合只追踪真实过程。这样既能覆盖考试，又不会为了“统一”制造第二套模糊知识。